

torch.logspace#

<https://pytorch.org/docs/stable/generated/torch.logspace.html#torch.logspace>

Description:

Creates a one-dimensional tensor of size `steps` whose values are evenly spaced from $\text{base}^{\text{start}}$ to base^{end} , inclusive, on a logarithmic scale with base `base`. That is, the values are: From PyTorch 1.11 `logspace` requires the `steps` argument. Use `steps=100` to restore the previous behavior. `start` (float or Tensor) – the starting value for the set of points. If Tensor, it must be 0-dimensional `end` (float or Tensor) – the ending value for the set of points. If Tensor, it must be 0-dimensional `steps` (int) – size of the constructed tensor

Function Signature

```
torch.logspace(start, end, steps, base=10.0, *, out=None,
dtype=None, layout=torch.strided, device=None,
requires_grad=False) → Tensor#
```

Parameters

Keyword Arguments

Parameters

Keyword

out (Tensor, optional) – the output tensor. dtype (torch.dtype, optional) – the data type to perform the computation in. Default: if None, uses the global default dtype (see `torch.get_default_dtype()`) when both start and end are real, and corresponding complex dtype when either is complex. layout (torch.layout, optional) – the desired layout of returned Tensor. Default: `torch.strided`. device (torch.device, optional) – the desired device of returned tensor. Default: if None, uses the current device for the default tensor type (see `torch.set_default_device()`). device will be the CPU for CPU tensor types and the current CUDA device for CUDA tensor types. requires_grad (bool, optional) – If autograd should record operations on the returned tensor. Default: False.

Code Examples

```
>>> torch.logspace(start=-10, end=10, steps=5)
tensor([ 1.0000e-10,  1.0000e-05,  1.0000e+00,  1.0000e+05,
  1.0000e+10])
>>> torch.logspace(start=0.1, end=1.0, steps=5)
tensor([ 1.2589,  2.1135,  3.5481,  5.9566, 10.0000])
>>> torch.logspace(start=0.1, end=1.0, steps=1)
tensor([1.2589])
>>> torch.logspace(start=2, end=2, steps=1, base=2)
tensor([4.0])
```

Detailed Documentation

Documentation

Creates a one-dimensional tensor of size `steps` whose values are evenly spaced from $\text{base}^{\text{start}}$ to base^{end} , inclusive, on a logarithmic scale with base `base`. That is, the values are:

From PyTorch 1.11 `logspace` requires the `steps` argument. Use `steps=100` to restore the previous behavior.

`start` (float or Tensor) – the starting value for the set of points. If Tensor, it must be 0-dimensional

`end` (float or Tensor) – the ending value for the set of points. If Tensor, it must be 0-dimensional

`steps` (int) – size of the constructed tensor

`base` (float, optional) – base of the logarithm function. Default: 10.0.

`out` (Tensor, optional) – the output tensor.

`dtype` (torch.dtype, optional) – the data type to perform the computation in. Default: if None, uses the global default dtype (see `torch.get_default_dtype()`) when both `start` and `end` are real, and corresponding complex dtype when either is complex.

`layout` (torch.layout, optional) – the desired layout of returned Tensor. Default: `torch.strided`.

`device` (`torch.device`, optional) – the desired device of returned tensor. Default: if None, uses the current device for the default tensor type (see `torch.set_default_device()`). device will be the CPU for CPU tensor types and the current CUDA device for CUDA tensor types.

`requires_grad` (bool, optional) – If autograd should record operations on the returned tensor. Default: False.